

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 50%

Duration : 1 hour

QUESTIONS: 2

Total Marks:40

Annexure A

Integrated Experiments

Code of Conduct:

I Kanish Kumar S(192511415)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Kanish Kumar S

Student 12: Kanishkumar S | Register Number: 192511415

INTEGRATED EXPERIMENT SCENARIO

Banking - Loan Default Prediction

Context: A bank wants an integrated pipeline that first predicts loan default risk from customer records (income, credit score, loan amount, repayment history) and then recommends an account-management action using a learning agent.

Question 1 - Decision Tree vs Neural Network Classifier

Implement a Decision Tree classifier and a Neural Network (Multi-Layer Perceptron) classifier on the same loan dataset. Compare their Accuracy, Precision, Recall and F1-Score. Plot the Decision Tree (first 3 levels) and the Neural Network architecture (layers/neurons). Justify which model you would deploy for real-time loan screening and why.

Question 2 - Reinforcement Learning for Account Management

Using the default-risk output from Experiment 1 as the initial state, implement a Q-Learning agent that recommends an account-management action (Approve / Reject / Restructure / Monitor) across simulated customer episodes. Show the Q-table after training, the learned policy, and integrate it with the classifier output into a single end-to-end decision pipeline (risk prediction -> action recommendation).

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Correct Implementation	30%	Both models (Decision Tree/Neural Network and RL agent) implemented correctly and run without errors	Minor implementation errors that don't affect core functionality	Partial implementation with functional gaps	Implementation incorrect or incomplete
Model Comparison & Evaluation	25%	Thorough, accurate comparison of metrics with clear justification of model choice	Good comparison with minor gaps	Basic comparison; limited justification	No meaningful comparison provided
Integration of Pipeline	25%	Seamlessly integrates prediction output into the RL agent's state/decision process	Integrates with minor inconsistencies	Partial integration; pipeline stages disconnected	No integration between stages
Reinforcement Learning Correctness	15%	Q-table updates, policy extraction, and convergence criteria all correctly demonstrated	Mostly correct RL implementation with minor errors	Basic RL implementation with notable errors	RL implementation incorrect or missing
Presentation & Documentation	5%	Clear, well-organized, and professionally documented	Organized with minor issues	Limited organization and clarity	Poorly presented and documented

Integrated Experiment Scenario

Banking – Loan Default Prediction

Complete solution covering Decision Tree vs Neural Network classification, performance comparison, visualization, Q-Learning for account management, and the integrated end-to-end decision pipeline.

Problem Statement

A bank wants an integrated pipeline that first predicts loan default risk from customer records such as income, credit score, loan amount and repayment history, and then recommends an account-management action using a learning agent.

Question 1 – Decision Tree vs Neural Network Classifier

Implement a Decision Tree classifier and a Neural Network (Multi-Layer Perceptron) classifier on the same loan dataset. Compare Accuracy, Precision, Recall and F1-Score. Plot the Decision Tree (first 3 levels) and Neural Network architecture (layers/neurons). Justify which model should be deployed for real-time loan screening.

1. Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix
)
```

2. Create / Load the Loan Dataset

If a CSV dataset is provided, use `pd.read_csv()`. Otherwise, the following code creates a simulated dataset for demonstration.

```
np.random.seed(42)

n = 500

income = np.random.randint(20000, 150000, n)
credit_score = np.random.randint(300, 850, n)
loan_amount = np.random.randint(5000, 100000, n)
repayment_history = np.random.randint(0, 11, n)

default_risk = (
    (credit_score < 600).astype(int) +
    (income < 40000).astype(int) +
    (loan_amount > 60000).astype(int) +
    (repayment_history < 5).astype(int)
)

loan_default = (default_risk >= 2).astype(int)

df = pd.DataFrame({
    "Income": income,
    "CreditScore": credit_score,
    "LoanAmount": loan_amount,
    "RepaymentHistory": repayment_history,
    "LoanDefault": loan_default
})

print(df.head())
print("Dataset Shape:", df.shape)
```

3. Split Features and Target

```
x = df[[
    "Income",
```

```

        "CreditScore",
        "LoanAmount",
        "RepaymentHistory"
    ])

y = df["LoanDefault"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

```

4. Decision Tree Classifier

```

dt_model = DecisionTreeClassifier(
    max_depth=5,
    random_state=42
)

dt_model.fit(X_train, y_train)
dt_pred = dt_model.predict(X_test)

dt_accuracy = accuracy_score(y_test, dt_pred)
dt_precision = precision_score(y_test, dt_pred, zero_division=0)
dt_recall = recall_score(y_test, dt_pred, zero_division=0)
dt_f1 = f1_score(y_test, dt_pred, zero_division=0)

print("Decision Tree Results")
print("Accuracy :", dt_accuracy)
print("Precision:", dt_precision)
print("Recall   :", dt_recall)
print("F1-Score :", dt_f1)

```

5. Plot Decision Tree – First 3 Levels

```

dt_plot = DecisionTreeClassifier(
    max_depth=3,
    random_state=42
)

dt_plot.fit(X_train, y_train)

plt.figure(figsize=(18, 10))
plot_tree(
    dt_plot,
    feature_names=X.columns,
    class_names=["No Default", "Default"],
    filled=True,
    rounded=True,
    fontsize=10
)

plt.title("Decision Tree - First 3 Levels")
plt.show()

```

6. Neural Network (MLP) Classifier

```

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

mlp_model = MLPClassifier(
    hidden_layer_sizes=(16, 8),
    activation="relu",
    solver="adam",
    max_iter=1000,
    random_state=42
)

mlp_model.fit(X_train_scaled, y_train)
mlp_pred = mlp_model.predict(X_test_scaled)

mlp_accuracy = accuracy_score(y_test, mlp_pred)
mlp_precision = precision_score(y_test, mlp_pred, zero_division=0)
mlp_recall = recall_score(y_test, mlp_pred, zero_division=0)
mlp_f1 = f1_score(y_test, mlp_pred, zero_division=0)

```

```

print("Neural Network Results")
print("Accuracy :", mlp_accuracy)
print("Precision:", mlp_precision)
print("Recall   :", mlp_recall)
print("F1-Score :", mlp_f1)

```

7. Compare Both Models

```

results = pd.DataFrame({
    "Model": ["Decision Tree", "Neural Network"],
    "Accuracy": [dt_accuracy, mlp_accuracy],
    "Precision": [dt_precision, mlp_precision],
    "Recall": [dt_recall, mlp_recall],
    "F1-Score": [dt_f1, mlp_f1]
})

print(results)

results.set_index("Model").plot(
    kind="bar",
    figsize=(10, 6)
)

plt.title("Decision Tree vs Neural Network")
plt.ylabel("Score")
plt.ylim(0, 1.1)
plt.xticks(rotation=0)
plt.grid(axis="y")
plt.show()

```

8. Neural Network Architecture

Input Layer: 4 neurons – Income, Credit Score, Loan Amount, Repayment History

Hidden Layer 1: 16 neurons

Hidden Layer 2: 8 neurons

Output Layer: 1 neuron – Default / No Default

```

layers = [
    ("Input", 4),
    ("Hidden 1", 16),
    ("Hidden 2", 8),
    ("Output", 1)
]

plt.figure(figsize=(10, 6))

for i, (name, neurons) in enumerate(layers):
    plt.scatter([i] * neurons, range(neurons), s=100)
    plt.text(
        i, neurons + 0.5,
        f"{name}\n({neurons} neurons)",
        ha="center",
        fontsize=11
    )

plt.xticks([])
plt.yticks([])
plt.title("Neural Network Architecture")
plt.show()

```

9. Model Deployment Justification

Recommended model: Decision Tree when its predictive performance is close to the MLP. A Decision Tree provides fast prediction, is easy to interpret, requires little preprocessing, and makes its decisions easier for banking staff to explain. If the Neural Network provides a substantially better Recall/F1-Score on a sufficiently large real dataset, it may be preferred provided interpretability and operational requirements can be addressed.

Question 2 – Reinforcement Learning for Account Management

Using the default-risk output from Experiment 1 as the initial state, implement a Q-Learning agent that recommends an account-management action across simulated customer episodes.

10. Define States and Actions

States: 0 = Low Risk, 1 = Medium Risk, 2 = High Risk

Actions: 0 = Approve, 1 = Reject, 2 = Restructure, 3 = Monitor

```
states = {
    0: "Low Risk",
    1: "Medium Risk",
    2: "High Risk"
}

actions = {
    0: "Approve",
    1: "Reject",
    2: "Restructure",
    3: "Monitor"
}

num_states = 3
num_actions = 4

Q = np.zeros((num_states, num_actions))

print("Initial Q-Table:")
print(Q)
```

11. Reward Table

```
reward_table = np.array([
    # Approve, Reject, Restructure, Monitor
    [10, -5, 3, 7], # Low Risk
    [2, 4, 8, 9], # Medium Risk
    [-10, 10, 8, 6] # High Risk
])

print(pd.DataFrame(
    reward_table,
    index=states.values(),
    columns=actions.values()
))
```

12. Q-Learning Training

The Q-Learning agent updates its Q-values using the current state, selected action, reward, and the best expected future value. The training uses simulated customer episodes.

```
alpha = 0.1
gamma = 0.9
epsilon = 1.0
epsilon_decay = 0.995
epsilon_min = 0.01
episodes = 1000

np.random.seed(42)

for episode in range(episodes):
    state = np.random.randint(0, num_states)

    for step in range(10):
        if np.random.random() < epsilon:
            action = np.random.randint(num_actions)
        else:
            action = np.argmax(Q[state])

        reward = reward_table[state, action]

        next_state = np.random.randint(0, num_states)

        Q[state, action] = Q[state, action] + alpha * (
            reward +
```

```

        gamma * np.max(Q[next_state]) -
        Q[state, action]
    )

    state = next_state

epsilon = max(
    epsilon_min,
    epsilon * epsilon_decay
)

```

13. Show Final Q-Table

```

Q_table = pd.DataFrame(
    Q,
    index=states.values(),
    columns=actions.values()
)

print("Final Q-Table:")
print(Q_table.round(2))

```

14. Learned Policy

```

policy = {}

for state in range(num_states):
    best_action = np.argmax(Q[state])
    policy[states[state]] = actions[best_action]

print("Learned Policy")

for state, action in policy.items():
    print(f"{state} -> {action}")

```

Expected Policy Pattern

Risk State	Likely Best Action
Low Risk	Approve
Medium Risk	Monitor / Restructure
High Risk	Reject

15. End-to-End Integration

The classifier first predicts whether the customer is likely to default. The result, together with risk-related information, is converted into a risk state. The Q-Learning agent then selects the account-management action with the highest learned Q-value.

```

def loan_decision_pipeline(
    income,
    credit_score,
    loan_amount,
    repayment_history
):
    customer = pd.DataFrame({
        "Income": [income],
        "CreditScore": [credit_score],
        "LoanAmount": [loan_amount],
        "RepaymentHistory": [repayment_history]
    })

    prediction = dt_model.predict(customer)[0]

    if prediction == 0 and credit_score >= 700:
        risk_state = 0
    elif prediction == 1 and credit_score < 600:
        risk_state = 2
    else:
        risk_state = 1

    action_index = np.argmax(Q[risk_state])
    recommended_action = actions[action_index]

    return {
        "Default Prediction":
            "Default" if prediction == 1 else "No Default",

```



```

        "Risk State": states[risk_state],
        "Recommended Action": recommended_action
    }
}

```

16. Test the Pipeline

```

# Customer 1 - Low risk example
result1 = loan_decision_pipeline(
    income=90000,
    credit_score=780,
    loan_amount=30000,
    repayment_history=9
)

print(result1)

# Customer 2 - High risk example
result2 = loan_decision_pipeline(
    income=30000,
    credit_score=520,
    loan_amount=80000,
    repayment_history=2
)

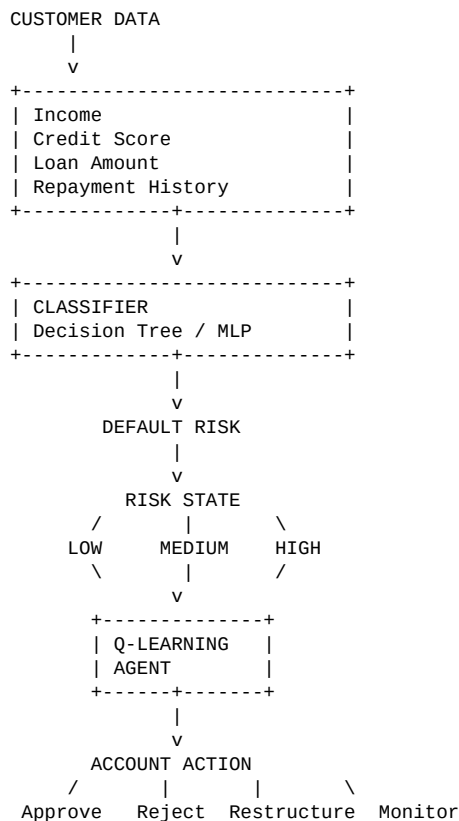
print(result2)

# Customer 3 - Medium risk example
result3 = loan_decision_pipeline(
    income=55000,
    credit_score=650,
    loan_amount=50000,
    repayment_history=5
)

print(result3)

```

17. End-to-End Architecture



18. Short Viva Explanation

First, customer information such as income, credit score, loan amount and repayment history is used to predict loan default using a Decision Tree and Neural Network. Their Accuracy, Precision, Recall and F1-Score are compared. The predicted risk is then converted into a Low, Medium or High Risk state. A Q-Learning agent learns the best account-management action for each state, such as Approve, Reject, Restructure or Monitor. Finally, both models

are integrated into an end-to-end pipeline that predicts loan risk and recommends an appropriate banking action.

Important Note

The dataset in this solution is simulated for educational purposes. For a real banking system, use a validated historical loan dataset, perform proper preprocessing and fairness checks, calibrate probabilities, and validate the model before deployment.